

Agentes de IA

De un agente hecho a mano a LangChain

Track de IA · Duoc UC Viña del Mar

¿Qué es un agente de IA?

Un sistema que usa un LLM para decidir qué pasos seguir y qué herramientas usar para cumplir un objetivo, repitiendo hasta terminarlo.



LLM solo

- Una pregunta, una respuesta
- Solo sabe lo que aprendió en su entrenamiento
- No puede consultar nada ni ejecutar acciones



Agente

- Descompone un objetivo en pasos
- Usa herramientas: APIs, bases de datos, funciones
- Observa el resultado y ajusta el plan
- Repite hasta cumplir el objetivo

Tres formas de construir un agente

1

Desde cero

Python puro. El modelo escribe qué herramienta quiere y tú lo extraes del texto.

2

Function calling

La API devuelve la herramienta pedida como JSON estructurado.

3

LangChain

Un framework arma el prompt, define herramientas y corre el ciclo por ti.

Cada etapa resuelve un problema de la anterior. Por eso vamos en este orden.

Las tres piezas de un agente



Cerebro

El LLM. Razona, planifica y decide qué hacer a continuación.



Memoria

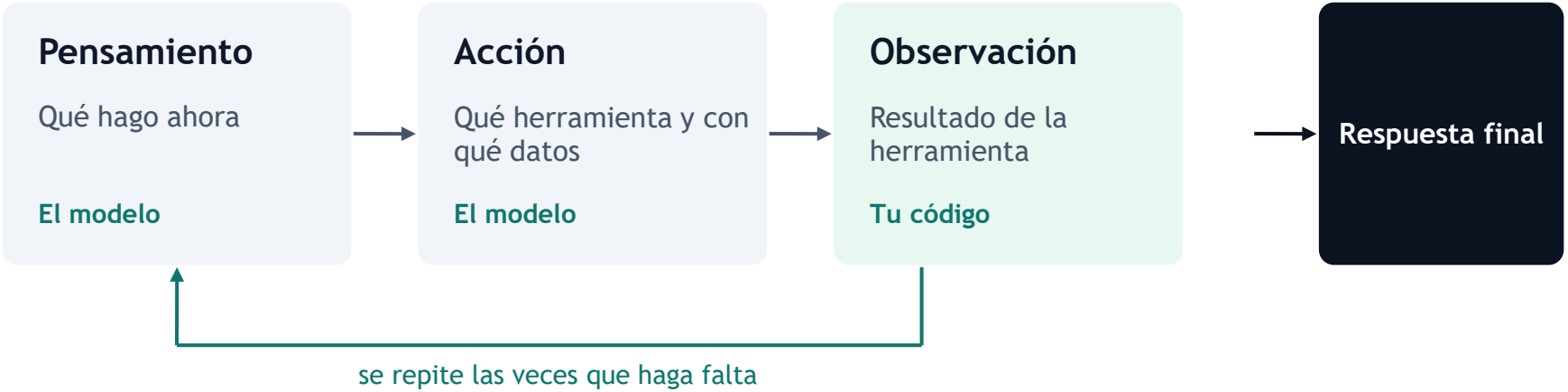
Corto plazo: la conversación actual. Largo plazo: interacciones pasadas.



Herramientas

Funciones o APIs que traen información o ejecutan acciones fuera del modelo.

El ciclo ReAct: razonar y actuar



Clave: el modelo nunca ve la herramienta ejecutarse. Solo lee la observación que tu código le devuelve.

Cómo funciona el agente hecho a mano

1	**Prompt de sistema**	Define el formato Pensamiento / Acción / Observación y describe las herramientas en texto.
2	**El modelo escribe**	Devuelve texto con una línea tipo Action y el JSON de la herramienta.
3	**Tu código extrae**	Busca la acción con expresiones regulares, la convierte a JSON y ejecuta la función.
4	**Vuelves a llamar**	Agregas la observación al historial y repites hasta ver Final Answer o un límite de vueltas.

Funciona, pero es frágil

Parsing frágil

Si el modelo cambia un poco el formato, la expresión regular no encuentra la acción.

Prompt a mano

Cada herramienta nueva implica reescribir y reformatear el prompt.

Ciclo repetitivo

El bucle, la condición de parada y el historial los programas tú.

Escala mal

Más herramientas, memoria o planes complejos se vuelven difíciles de mantener.

2 Etapa 2: function calling

De pedir en texto a pedir en JSON

Antes

El modelo escribe su intención como texto libre y tú la tienes que interpretar.

```
Action: {"tool": "search_web", ...}
```

Ahora

La respuesta trae un campo tool_calls con el nombre de la función y sus argumentos.

```
tool_calls → name + arguments
```

Fiable

El modelo casi siempre genera JSON válido.

Sin regex

Ya no parseas texto: lees campos.

Depurable

Ves exactamente qué pidió y con qué datos.

Cómo se describe una herramienta

`name`	Cómo se llama. Debe coincidir con tu función real.
`description`	Lo que el modelo lee para decidir si usarla. La parte más importante.
`parameters`	Qué argumentos recibe y de qué tipo (JSON Schema).
`required`	Cuáles de esos argumentos son obligatorios.

Se envía en la llamada junto con los mensajes, en el parámetro tools.

Dos cosas distintas

La función de Python vive en tu código.

La descripción es lo único que ve el modelo.

Si la descripción es mala, el modelo elige mal.

2 Etapa 2: function calling

El flujo completo, paso a paso

1	**Tu app → modelo**	Pregunta del usuario + lista de herramientas
2	**Modelo → tu app**	En vez de texto, devuelve tool_calls
3	**Tu app**	Ejecuta la función de Python que el modelo pidió
4	**Tu app → modelo**	Envía el resultado como mensaje con role "tool"
5	**Modelo → usuario**	Con esa información, redacta la respuesta final

El modelo nunca ejecuta nada.

Solo pide una herramienta. Tu código decide si la ejecuta, la ejecuta y le devuelve el resultado.

Igual que con el historial: todo lo que pasa fuera del modelo es responsabilidad de tu aplicación.

Qué deja de ser tu problema

Antes lo hacías tú	Con LangChain
Escribir el JSON Schema de cada herramienta	Decorador @tool: el esquema sale de la firma y el docstring
Armar el prompt del agente a mano	Prompt prediseñado y probado, descargado del hub
El bucle, el if/else y la parada	AgentExecutor corre el ciclo completo

Agente vs. AgentExecutor

Agente

Une LLM + herramientas + prompt.

Decide qué herramienta usar, pero no puede ejecutar el ciclo por sí solo.

AgentExecutor

Corre el ciclo: llama al agente, ejecuta la herramienta, le devuelve el resultado y repite hasta la respuesta final.

El historial (chat_history) lo sigues pasando tú en cada invoke. El executor lo usa, no lo guarda.

Los tres enfoques lado a lado

	Desde cero	Function calling	LangChain
Cómo pide la herramienta	Texto que tú parseas	JSON en tool_calls	JSON, lo maneja el framework
Cómo defines la herramienta	Descripción dentro del prompt	JSON Schema a mano	@tool + docstring
Quién corre el ciclo	Tu bucle	Tu código	AgentExecutor
Sirve para	Entender el mecanismo	Control total, sin framework	Avanzar rápido y escalar

Notebooks

1. `1-agent-fundamentals.ipynb`
2. `2-agent-function-calling.ipynb`
3. `3-langchain-agent.ipynb`